

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-afd3b09e5cbadf5a8.jsonl`  
- SHA-256: `3c8e3adc5b388b1f2bbbd4c31e9efdd2901cd87ca729bdf79b2f1d6d99c64483`  
- Source modified: `2026-07-04T00:45:22+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `subagents`  
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

Transcript

1. user (2026-07-04T00:44:01.418Z)

Code-review finder. Target: PR #458 of badminton-highlight-indexer. Get the diff: run `cd C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer && gh pr diff 458`. The PR branch is `origin/codex/a7-a9-quality-chain`; read PR-branch file contents with `git show origin/codex/a7-a9-quality-chain:<path>` (the working checkout is on master, NOT the PR branch). The two changed code files are also extracted at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458_fusion_golden.py and pr458_fusion_audio.py.

This PR adds A7/A8/A9 quality tooling to backend/eval/fusion_golden.py + fusion_audio.py: source-manifest video-path attach, --only filter, --missing-fusion-only, --max-gpu-temp-c abort, full-fusion schema detection, plus doc + ablation-test updates.

ANGLE A ? line-by-line. Read every added hunk, then the enclosing function. For each line ask what input/state makes it wrong. Scrutinize specifically:

1. `is\_full\_fusion\_feature\_file`: the `if not candidates: return True` branch ? an existing windowing-only JSON with an EMPTY candidates list is classified as FULL-fusion ? run()'s resume would SKIP re-extracting it (stale file kept). Is that a real misclassification? Also the `all(... "shuttle" in candidate and "person" in candidate ...)` ? does it check the values are non-null/non-empty, or just key presence?
2. run()'s resume logic (the `existing`/`missing\_fusion\_only` branches): trace all combinations of (fresh, missing_fusion_only, existing, existing_full). Is any branch unreachable or does any wrong combo skip a video that should be re-extracted, or re-extract one that shouldn't?
3. `\_progress`: changed to `rec = telemetry.snapshot(...)` then reads `rec.get("gpu")`. Does `telemetry.snapshot` actually RETURN a dict with a "gpu" key? Grep the telemetry class (backend/utils/run_telemetry.py or similar ? find `def snapshot`). If snapshot returns None, the max_gpu_temp_c abort silently never fires ? the safety feature is dead. Confirm the return contract.
4. `load\_source\_video\_paths`: `entry.get("source\_status", "present\_local")` defaults ABSENT status to present_local (attaches path for entries with no status). And `attach\_source\_video\_paths` mutates spec dicts in place. Any correctness issue?
5. fusion_audio.augment: the `FileNotFoundError` raise is INSIDE the per-spec loop ? one missing video aborts the whole batch after partial work. Intended or a footgun?
6. `\_filter\_specs` dedup + missing check ? correct?

Return up to 6 candidates as JSON [{file, line, summary, failure_scenario}]. line = line in the PR-branch file. Concrete scenarios only. [] if none. Final message = only the JSON array.

2. assistant (2026-07-04T00:44:04.618Z)

I'll analyze PR #458 systematically. Let me start by getting the diff and examining the changed files.

3. user (2026-07-04T00:44:06.557Z)

<persisted-output>
Output too large (51.9KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\tool-results\b1legj8g2.txt

Preview (first 2KB):
diff --git a/SESSION_HANDOFF.md b/SESSION_HANDOFF.md
index 31ea4c14..a50cc8a1 100644
--- a/SESSION_HANDOFF.md
+++ b/SESSION_HANDOFF.md
@@ -2,49 +2,51 @@

You are here

-A6 is implemented and open for review as PR #455:
-`https://github.com/Khelsutra/badminton-highlight-indexer/pull/455`.
+A6 PR #455 is merged. This checkout is on `codex/a7-a9-quality-chain`, fast-forwarded to
+`origin/master` at `c5fc27c`, with tracked edits in:

-The branch is `codex/a6-ablation-cli`, rebased onto `origin/master` after A5
-merged as `c8eec7c` / PR #454. It fixes the `python -m backend.eval.ablation`
-circular import by making `ablation.export\_pdf` a lazy wrapper, adds a
-module-entrypoint regression test, and records the n=15 R2/tolF1 contrast in
-`docs/ALPHA\_LAUNCH\_READINESS.md`.
+- `backend/eval/fusion\_golden.py`
+- `backend/eval/fusion\_audio.py`
+- `tests/test\_fusion\_compare.py`
+- `tests/test\_audio\_features.py`
+- `docs/ALPHA\_LAUNCH\_READINESS.md`
+- `SESSION\_HANDOFF.md`

-Validation completed locally on 2026-07-03:
-`ruff check .`, `python -m mpy backend training`, and
-`python run\_all\_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`
-all passed. Full suite result: 1609 passed, 6 skipped, 85.65% coverage.
+A7/A8/A9 have been run locally against the 15-video corpus, and draft PR #458 is open:
+`https://github.com/Khelsutra/badminton-highlight-indexer/pull/458`. The generated
+feature/model artifacts live under ignored `output/` and `artifacts/` paths and should not
+be committed.

Next steps / open threads

-1. Review/merge PR #455 for A6.
-2. A7 is the next indexer-owned quality item: regenerate or upconvert
- full-fusion feature JSONs for the newer nine golden videos so fusion/person
- /audio analyses can run on all 15 videos. GPU is approved on this machine.
-3. A8 follows A7: rerun `fusion\_compare`, group ablation, and served contrast;
- explicitly promote or reject fusion/person/Gate-A served-default decisions.
...
</persisted-output>

4. assistant (2026-07-04T00:44:08.389Z)

Let me read the full diff and the extracted files:

5. user (2026-07-04T00:44:09.616Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer.

6. user (2026-07-04T00:44:10.259Z)

1 """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2

```

3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12      python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17
18      from __future__ import annotations
19
20      import argparse
21      import json
22      import os
23      import time
24      from typing import Any, Dict, List, Optional, Sequence
25
26      from backend.eval import distill_local
27      from backend.eval.calibrate_wasb import load_golden_gts
28      from backend.eval.golden_manifest import load_resolved_manifest
29      from backend.eval.training import label_candidates
30      from backend.pipeline.detectors import fusion_features as ff
31      from backend.pipeline.detectors import rally_gate
32      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33      from backend.utils.run_telemetry import RunTelemetry
34
35      # The 5 fps/resolution-invariant shuttle features distill_local already computes.
36      SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
37      # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
38      PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
39      SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
40
41
42      def is_full_fusion_feature_file(path: str) -> bool:
43          """True when an existing golden feature JSON already has shuttle+person cue dicts.
44
45          Corpus-growth runs can leave leaner windowing-only JSONs with the same filename
pattern.
46          Those are useful for gts/windowing analysis, but they are not the replayable fusion
47          substrate that ``fusion_compare`` and ``ablation`` need.
48          """
49
50          try:
51              with open(path, encoding="utf-8") as f:
52                  data = json.load(f)
53          except (OSError, json.JSONDecodeError, TypeError):
54              return False
55          candidates = data.get("candidates")
56          if not isinstance(candidates, list):
57              return False
58          if not candidates:

```

```

59         return True
60     return all(
61         isinstance(candidate, dict)
62         and "shuttle" in candidate
63         and "person" in candidate
64         for candidate in candidates
65     )
66
67
68 def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
69     """Load ``video_id -> local_path`` from the collector golden-source manifest.
70
71     The collector records source/proxy ownership and full-file MD5s. Only entries marked
72     ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
73     """
74
75     if not source_manifest_path:
76         return {}
77     with open(source_manifest_path, encoding="utf-8") as f:
78         data = json.load(f)
79     entries = data.get("videos", data) if isinstance(data, dict) else data
80     if not isinstance(entries, list):
81         raise ValueError(f"source manifest must contain a videos list: {source_manifest_path}")
82
83     paths: Dict[str, str] = {}
84     for entry in entries:
85         if not isinstance(entry, dict):
86             continue
87         video_id = entry.get("video_id") or entry.get("name")
88         local_path = entry.get("local_path")
89         if not video_id or not local_path:
90             continue
91         if entry.get("source_status", "present_local") != "present_local":
92             continue
93         paths[str(video_id)] = str(local_path)
94     return paths
95
96
97 def attach_source_video_paths(
98     specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
99 ) -> int:
100     """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
101
102     Returns the number of specs updated. The collector source manifest is authoritative
103     for local source/proxy video location and overrides derived/stale manifest paths.
104     """
105
106     source_paths = load_source_video_paths(source_manifest_path)
107     updated = 0
108     for spec in specs:
109         name = str(spec.get("name", ""))
110         source_path = source_paths.get(name)
111         if not source_path:
112             continue
113         if spec.get("video_path") == source_path:
114             continue
115         spec["video_path"] = source_path
116         updated += 1
117     return updated
118
119
120 def _filter_specs(
121     specs: Sequence[Dict[str, Any]], only: Sequence[str]
122 ) -> List[Dict[str, Any]]:

```

```

123         if not only:
124             return list(specs)
125         wanted = list(dict.fromkeys(only))
126         wanted_set = set(wanted)
127         filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
128         found = {str(spec.get("name")) for spec in filtered}
129         missing = [name for name in wanted if name not in found]
130         if missing:
131             raise ValueError(f"--only requested unknown video(s): {' ', '.join(missing)}")
132         return filtered
133
134
135     def _derive_video_path(spec: Dict[str, Any]) -> str:
136         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
137         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
138         convention)."""
139         if spec.get("video_path"):
140             return str(spec["video_path"])
141         labels = str(spec["labels"])
142         if labels.endswith(".rallies.csv"):
143             return labels[: -len(".rallies.csv")] + ".MP4"
144         raise ValueError(
145             f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv"
146         )
147
148     def extract_video(
149         spec: Dict[str, Any],
150         detector: Optional[Any] = None,
151         frame_skip: int = 3,
152         iou: float = 0.5,
153         log_every_sec: float = 15.0,
154         telemetry: Optional[Any] = None,
155         use_gpu: bool = True,
156         detector_model: Optional[str] = None,
157         max_gpu_temp_c: Optional[float] = None,
158     ) -> Dict[str, Any]:
159         """One golden video -> a feature record (shuttle + person features per candidate
160         window).
161
162         Returns `{name, fps, frame_width,
163         candidates:[{start,end,shuttle{ },person{ },label}], gts}``.
164         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
165         tests;
166         when None a real `PersonDetector` is built (loads torchvision on first use).
167         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
168         snapshots
169         from the detection progress callback so a hard kill leaves a forensic runlog.
170         """
171         fps, fw = float(spec["fps"]), float(spec["frame_width"])
172         traj = str(spec["trajectory"])
173         points = TrackNetRunner.parse_trajectory_csv(traj)
174         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
175         gts = load_golden_gts(str(spec["labels"]))
176         labels = label_candidates(intervals, gts, iou)
177         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
178         estimate
179         # (parity with the shuttle net-crossing gate) rather than hard-coding an
180         orientation.
181         axis, net_px, _span = rally_gate.estimate_play_axis(points)
182         video_path = _derive_video_path(spec)
183         if not os.path.isfile(video_path):
184             raise FileNotFoundError(
185                 f"{spec.get('name')}: source video not found for fusion extraction:
186                 {video_path}"

```

```

180         )
181
182     if detector is None:
183         from backend.pipeline.detectors.person_detector import PersonDetector
184
185         indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}
186         if detector_model:
187             indexing_cfg["detector_model"] = detector_model
188         detector = PersonDetector({"indexing": indexing_cfg})
189
190     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
191     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
192     n = len(intervals)
193     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
194     print(
195         f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...",
196         flush=True,
197     )
198     t0 = time.monotonic()
199     state = {"last": t0}
200
201     def _progress(done: int, total: int) -> None:
202         rec = None
203         if telemetry is not None:
204             rec = telemetry.snapshot(done, total, phase=spec["name"])
205         if max_gpu_temp_c is not None and rec is not None:
206             gpu = rec.get("gpu") or {}
207             temp = gpu.get("temp_c")
208             if temp is not None and float(temp) >= max_gpu_temp_c:
209                 raise RuntimeError(
210                     f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
211                     f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal
shutdown"
212                 )
213         now = time.monotonic()
214         if now - state["last"] >= log_every_sec or done >= total:
215             el = now - t0
216             rate = done / el if el > 0 else 0.0
217             eta = (total - done) / rate if rate > 0 else 0.0
218             pct = 100 * done // total if total else 0
219             print(
220                 f"[fusion_golden]   {spec['name']}: detect frame {done}/{total} ({pct}%)
"
221                 f"| {el:.0f}s elapsed | ETA {eta:.0f}s",
222                 flush=True,
223             )
224             state["last"] = now
225
226     per_window = detector.detections_over_windows(
227         video_path, win_frames, frame_skip, progress_cb=_progress
228     )
229
230     candidates: List[Dict[str, Any]] = []
231     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
232         det = per_window[i]
233         det_fw = det.get("frame_width") or fw
234         det_fh = det.get("frame_height") or 0.0
235         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
236         # normalisation fusion_features uses (x/width, y/height).
237         if axis == "x":
238             net_norm = net_px / det_fw if det_fw > 0 else 0.5
239         else:
240             net_norm = net_px / det_fh if det_fh > 0 else 0.5

```

```

241     sf, ef = win_frames[i]
242     shuttle_in = [p for p in points if sf <= p.frame <= ef]
243     person = ff.compute_person_features(
244         det["frames"],
245         shuttle_in,
246         det_fw,
247         det_fh,
248         court_polygon=None,
249         net=(axis, net_norm),
250     )
251     candidates.append(
252         {
253             "start": float(s),
254             "end": float(e),
255             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
256             "person": {k: float(v) for k, v in person.items()},
257             "label": int(labels[i]),
258         }
259     )
260     return {
261         "name": spec["name"],
262         "fps": fps,
263         "frame_width": fw,
264         "candidates": candidates,
265         "gts": [[float(a), float(b)] for a, b in gts],
266     }
267
268
269 def run(
270     manifest_path: str,
271     out_dir: str,
272     frame_skip: int = 3,
273     iou: float = 0.5,
274     fresh: bool = False,
275     smallest_first: bool = False,
276     source_manifest: Optional[str] = None,
277     only: Sequence[str] = (),
278     missing_fusion_only: bool = False,
279     use_gpu: bool = True,
280     detector_model: Optional[str] = None,
281     max_gpu_temp_c: Optional[float] = None,
282 ) -> List[str]:
283     specs = load_resolved_manifest(manifest_path)
284     source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
285     if source_manifest:
286         updated = attach_source_video_paths(specs, source_manifest)
287         print(
288             f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
289             f"from {source_manifest}",
290             flush=True,
291         )
292     specs = _filter_specs(specs, only)
293     if smallest_first:
294         # Process small videos first for quick feedback (the slow full match lands
295         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
296         # row/frame).
297         specs = sorted(
298             specs,
299             key=lambda s: (
300                 os.path.getsize(s["trajectory"])
301                 if os.path.isfile(s.get("trajectory", ""))
302                 else 0
303             ),
304         )

```

```

304     os.makedirs(out_dir, exist_ok=True)
305     # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
306     # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
307     # break extraction, so guard the build and fall back to no telemetry on any error.
308     telem: Optional[RunTelemetry]
309     try:
310         telem = RunTelemetry(
311             os.path.join(out_dir, "run-telemetry.jsonl"), label="fusion-golden"
312         )
313     except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
314         telem = None
315     written: List[str] = []
316     for vi, spec in enumerate(specs, 1):
317         out = os.path.join(out_dir, f"{spec['name']}_golden-features.json")
318         existing = os.path.isfile(out)
319         existing_full = existing and is_full_fusion_feature_file(out)
320         # Resumable: skip a video whose full-fusion JSON already exists. A leaner
321         # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
322         if missing_fusion_only and not fresh and existing_full:
323             print(
324                 f"[fusion-golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
325                 f"features already present -> {out}",
326                 flush=True,
327             )
328             written.append(out)
329             continue
330         if not fresh and existing and existing_full:
331             print(
332                 f"[fusion-golden] ({vi}/{len(specs)}) {spec['name']}: already extracted
"
333                 f"-> {out} (use --fresh to redo)",
334                 flush=True,
335             )
336             written.append(out)
337             continue
338         if not fresh and existing and not existing_full:
339             print(
340                 f"[fusion-golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
341                 f"is not full-fusion schema; re-extracting...",
342                 flush=True,
343             )
344             print(
345                 f"[fusion-golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
346                 flush=True,
347             )
348             t0 = time.monotonic()
349             if telem is not None:
350                 telem.event("video_start", spec["name"])
351             try:
352                 data = extract_video(
353                     spec,
354                     frame_skip=frame_skip,
355                     iou=iou,
356                     telemetry=telem,
357                     use_gpu=use_gpu,
358                     detector_model=detector_model,
359                     max_gpu_temp_c=max_gpu_temp_c,
360                 )
361             except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
362                 if telem is not None:
363                     telem.event("error", spec["name"], err=str(e))

```

```

364         raise
365     n_pos = sum(c["label"] for c in data["candidates"])
366     with open(out, "w", encoding="utf-8") as f:
367         json.dump(data, f, indent=2)
368     if telem is not None:
369         telem.event(
370             "video_done",
371             spec["name"],
372             windows=len(data["candidates"]),
373             positives=n_pos,
374         )
375     print(
376         f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
377         f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
378         f"-> {out}",
379         flush=True,
380     )
381     written.append(out)
382     return written
383
384
385 def main() -> None:
386     ap = argparse.ArgumentParser(
387         description="Extract golden-set shuttle+person features (fusion P3 GPU step)."
388     )
389     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
390     ap.add_argument("--out-dir", default="output")
391     ap.add_argument("--frame-skip", type=int, default=3)
392     ap.add_argument("--iou", type=float, default=0.5)
393     ap.add_argument(
394         "--fresh", action="store_true", help="re-extract even if a JSON already exists"
395     )
396     ap.add_argument(
397         "--smallest-first",
398         action="store_true",
399         help="process smaller videos first for quick feedback (LOVO is order-
independent)",
400     )
401     ap.add_argument(
402         "--source-manifest",
403         default=None,
404         help=(
405             "collector golden_source_videos.json; overrides video paths for source/proxy
"
406             f"videos (or set {SOURCE_MANIFEST_ENV})"
407         ),
408     )
409     ap.add_argument(
410         "--only",
411         action="append",
412         default=[],
413         metavar="VIDEO_ID",
414         help="process one manifest video by name; repeatable",
415     )
416     ap.add_argument(
417         "--missing-fusion-only",
418         action="store_true",
419         help="skip videos whose existing JSON already has full shuttle+person fusion
schema",
420     )
421     ap.add_argument(
422         "--cpu",
423         action="store_true",

```

```

424         help="run torchvision person detection on CPU instead of CUDA",
425     )
426     ap.add_argument(
427         "--detector-model",
428         default=None,
429         choices=[
430             "fasterrcnn_resnet50_fpn",
431             "fasterrcnn_mobilenet_v3_large_fpn",
432             "retinanet_resnet50_fpn",
433         ],
434         help="torchvision detector model; default keeps the configured Faster R-CNN
ResNet",
435     )
436     ap.add_argument(
437         "--max-gpu-temp-c",
438         type=float,
439         default=None,
440         help="abort extraction if telemetry sees GPU temperature at or above this
value",
441     )
442     args = ap.parse_args()
443     run(
444         args.manifest,
445         args.out_dir,
446         args.frame_skip,
447         args.iou,
448         fresh=args.fresh,
449         smallest_first=args.smallest_first,
450         source_manifest=args.source_manifest,
451         only=args.only,
452         missing_fusion_only=args.missing_fusion_only,
453         use_gpu=not args.cpu,
454         detector_model=args.detector_model,
455         max_gpu_temp_c=args.max_gpu_temp_c,
456     )
457
458
459     if __name__ == "__main__":
460         main()
461

```

7. user (2026-07-04T00:44:10.492Z)

```

1     """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
2     only).
3     Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
4     runs
5     `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
6     features per
7     window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
8     variant
9     and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
10    TOP
11    of person, not against raw shuttle. Audio enters only as columns the learned scorer
12    weights
13    (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
14    Run (no GPU):
15    python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
16    """
17
18    from __future__ import annotations
19

```

```

16     import argparse
17     import json
18     import logging
19     import os
20     from typing import Any, Dict, List, Optional, Sequence
21
22     from backend.eval import experiment
23     from backend.eval.fusion_compare import delta_stats
24     from backend.eval.fusion_golden import (
25         PERSON_FEATURE_NAMES,
26         SHUTTLE_FEATURE_NAMES,
27         SOURCE_MANIFEST_ENV,
28         _derive_video_path,
29         _filter_specs,
30         attach_source_video_paths,
31     )
32     from backend.eval.golden_manifest import load_resolved_manifest
33     from backend.pipeline.audio.hit_detector import detect_hits_in_video
34     from backend.pipeline.detectors import audio_features as af
35
36     logger = logging.getLogger(__name__)
37
38
39     def augment(
40         manifest_path: str,
41         features_dir: str,
42         k: float = 2.5,
43         source_manifest: Optional[str] = None,
44         only: Sequence[str] = (),
45     ) -> List[str]:
46         """Add the 3 audio features to each existing ``<name>_golden_features.json``
47         window."""
48         specs = load_resolved_manifest(manifest_path)
49         source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
50         if source_manifest:
51             updated = attach_source_video_paths(specs, source_manifest)
52             print(
53                 f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
54                 f"from {source_manifest}",
55                 flush=True,
56             )
57         specs = _filter_specs(specs, only)
58         written: List[str] = []
59         for spec in specs:
60             jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
61             if not os.path.isfile(jp):
62                 print(
63                     f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
64                     first; skipping"
65                 )
66                 continue
67             video = _derive_video_path(spec)
68             if not os.path.isfile(video):
69                 raise FileNotFoundError(
70                     f"{spec.get('name')}: source video not found for audio augmentation:
71                     {video}"
72                 )
73             print(
74                 f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
75                 flush=True,
76             )
77             hits = detect_hits_in_video(video, k=k)
78             with open(jp, encoding="utf-8") as f:
79                 data = json.load(f)
80                 for c in data["candidates"]:

```

```

78         c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
79         with open(jp, "w", encoding="utf-8") as f:
80             json.dump(data, f, indent=2)
81         print(
82             f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows",
83             flush=True,
84         )
85         written.append(jp)
86         return written
87
88
89     def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
90         """Load the FULL-FUSION-SCHEMA feature JSONs into VideoCandidates including the
audio
91         columns (requires `augment` to have run ? raises if a fusion JSON lacks audio).
92
93         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts) are
SKIPPED, not
94         crashed on, so a mixed corpus loads the fusion subset cleanly (see
``_is_fusion_schema``).
95         The skip is checked BEFORE the audio-presence guard, so a genuine fusion file that
simply
96         hasn't been ``--augment``-ed still raises rather than being silently dropped."""
97         return experiment.load_golden_videos(
98             features_dir,
99             shuttle_columns=SHUTTLE_FEATURE_NAMES,
100             person_columns=PERSON_FEATURE_NAMES,
101             audio_columns=af.AUDIO_FEATURE_NAMES,
102             audio_policy="require",
103             skip_logger=logger,
104             skip_message="fusion_audio.load_videos_with_audio: skipping %s ? windowing-only
"
105             "schema (no shuttle/person cue dicts), not a fusion-feature file",
106             no_audio_error="{path} has no audio features ? run ``--augment`` first",
107         )
108
109
110     def compare_incremental(
111         videos: List[experiment.VideoCandidates], iou: float = 0.5, threshold: float = 0.5
112     ) -> Dict[str, Any]:
113         """Incremental audio lift: shuttle+person vs shuttle+person+audio (honest
LOVO)."""
114         person = experiment.Variant(
115             name="shuttle_person",
116             feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
117             threshold=threshold,
118         )
119         person_audio = experiment.Variant(
120             name="shuttle_person_audio",
121             feature_names=list(SHUTTLE_FEATURE_NAMES)
122             + list(PERSON_FEATURE_NAMES)
123             + list(af.AUDIO_FEATURE_NAMES),
124             threshold=threshold,
125         )
126         return experiment.compare(videos, person, person_audio, iou=iou, honest=True)
127
128
129     def format_result(res: Dict[str, Any], iou: float) -> str:
130         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
131         ds = delta_stats(res)
132         st = ds["sign_test"]
133         lines = [
134             f"=== Fusion P4 incremental AUDIO LOVO (n={res['num_videos']}, IoU>={iou},
honest) ===",

```

```

135         f" shuttle+person          : F1={a['f1']:.3f} P={a['precision']:.3f}
R={a['recall']:.3f}",
136         f" shuttle+person+audio : F1={b['f1']:.3f} P={b['precision']:.3f}
R={b['recall']:.3f}",
137         f" incremental ?F1 = {d['f1']:+.3f} ({res['delta_pct']['f1']:+.1f}%)",
138         f" paired ?F1 {ds['mean_delta']:+.3f} [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}] "
139         f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f}) "
140         f"| CI clears 0: {ds['ci_excludes_zero']}",
141     ]
142     for pv in res["per_video_delta"]:
143         lines.append(f" {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
144     return "\n".join(lines)
145
146
147 def main() -> None:
148     ap = argparse.ArgumentParser(
149         description="Fusion P4 ? incremental audio lift on the golden set."
150     )
151     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
152     ap.add_argument("--features-dir", default="output")
153     ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
154     ap.add_argument(
155         "--source-manifest",
156         default=None,
157         help=(
158             "collector golden_source_videos.json; overrides video paths for source/proxy
"
159             f"videos (or set {SOURCE_MANIFEST_ENV})"
160         ),
161     )
162     ap.add_argument(
163         "--only",
164         action="append",
165         default=[],
166         metavar="VIDEO_ID",
167         help="augment one manifest video by name; repeatable",
168     )
169     ap.add_argument(
170         "--augment",
171         action="store_true",
172         help="add audio features to the existing JSONs",
173     )
174     ap.add_argument(
175         "--compare", action="store_true", help="report the incremental audio LOVO ?F1"
176     )
177     ap.add_argument("--iou", type=float, default=0.5)
178     ap.add_argument("--record", action="store_true")
179     args = ap.parse_args()
180
181     if args.augment:
182         augment(
183             args.manifest,
184             args.features_dir,
185             args.k,
186             source_manifest=args.source_manifest,
187             only=args.only,
188         )
189     if args.compare:
190         videos = load_videos_with_audio(args.features_dir)
191         if len(videos) < 2:
192             raise SystemExit(f"need >=2 augmented feature files in {args.features_dir}")
193         res = compare_increment(videos, iou=args.iou)
194         print(format_result(res, args.iou))
195         if args.record:

```

```

196         row = experiment.record_experiment(res)
197         print(
198             f"recorded -> eval_baselines/experiment_leaderboard.json
199         ({row['timestamp']})"
200     )
201
202     if __name__ == "__main__":
203         main()
204

```

8. assistant (2026-07-04T00:44:12.667Z)

Now let me get the telemetry class to verify the snapshot() contract:

9. user (2026-07-04T00:44:14.243Z)

```

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/utlis/run_telemetry.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/tests/test_fusion_golden_telemetry.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/tests/test_run_telemetry.py

```

10. user (2026-07-04T00:44:16.050Z)

```

1     """Structured run telemetry ? a compact, rotating JSONL "black box" for long/flaky runs.
2
3     A long GPU/CV run (the fusion-golden extraction, a WSL/WASB pass, an ffmpeg stitch) can
4     die with
5     **no Python traceback** ? the OS kills it (a console-control event, an OOM, a thermal
6     event). When
7     that happens the only forensic evidence is whatever was last flushed to disk. This
8     module keeps a
9     tiny (~10 KB) rotating runlog of *execution velocity + machine health* so a post-mortem
10    is a
11    bullseye instead of a guess: "it was at 26 fps, GPU 85 °C throttled to 1125/2100 MHz,
12    4.8 GB VRAM
13    free, CPU 60 %, RAM 21/32 GB ? then vanished."
14
15    It is the **observability** companion to the `ExecutionPolicy` *control* layer
16    (`docs/EXECUTION_POLICY.md`): the policy decides wait / abort / resume; this records the
17    signals
18    that explain *why* a run slowed or died, and lets us compare execution velocity across
19    quality
20    iterations.
21
22    Design:
23    - **Caller-driven snapshots** (no background thread): call ``snapshot(done, total)``
24    from an
25    existing progress callback; fps is derived from the delta since the previous snapshot.
26    ``event(kind, msg)`` records discrete moments (start, video-done, error) WITH a health
27    sample, so
28    the last line before a kill captures the machine state at death.
29    - **Capped + rotating**:: a fixed ``meta`` header line (machine / GPU / start) is always
30    kept; recent
31    snapshot/event lines form an on-disk ring trimmed to ``max_bytes`` (~10 KB). The file
32    is rewritten
33    atomically (tmp + ``os.replace``) on each write, so a hard kill never leaves it half-
34    written.
35    - **Graceful degradation**:: GPU stats come from ``nvidia-smi`` (absent -> ``None``);
36    system stats
37    from ``psutil`` (not installed -> ``None``). Collection never raises into the caller ?
38    a telemetry
39    hiccup must not break the run it observes.

```

```

26     - **Deterministic / testable**: the clock and the two collectors are injectable, so fps,
rotation,
27     and parsing are unit-tested with no real GPU, no psutil, and no wall-clock.
28
29     All stdlib + optional ``psutil`` (BSD-3, commercial-clean). No network, no training-data
path.
30     """
31
32     from __future__ import annotations
33
34     import json
35     import os
36     import shutil
37     import subprocess
38     import time
39     from typing import Any, Callable, Dict, List, Optional
40
41     GpuFn = Callable[[], Optional[Dict[str, Any]]]
42     SysFn = Callable[[], Optional[Dict[str, Any]]]
43
44     # nvidia-smi fields pulled in one query, in this order (see gpu_stats parsing).
45     _GPU_QUERY =
"temperature.gpu,utilization.gpu,power.draw,clocks.sm,clocks.max.sm,memory.used,memory.free"
46     _THROTTLE_CLOCK_FRAC = 0.8 # SM clock below this fraction of max => flagged throttled
47     _THROTTLE_TEMP_C = (
48         84.0 # ... or temperature at/above this (laptop Max-Q throttle territory)
49     )
50
51
52     def _num(s: str) -> Optional[float]:
53         try:
54             return float(s)
55         except (TypeError, ValueError):
56             return None
57
58
59     def gpu_stats(timeout: float = 4.0) -> Optional[Dict[str, Any]]:
60         """One ``nvidia-smi`` sample as a dict, or ``None`` if nvidia-smi is unavailable /
errors.
61
62         Returns util %, temperature, power (W), current + max SM clock (MHz), VRAM used /
free (MB), and a
63         derived ``throttled`` flag (SM clock well below max, or hot). Pure read; never
raises.
64         """
65         exe = shutil.which("nvidia-smi")
66         if not exe:
67             return None
68         try:
69             out = subprocess.run(
70                 [exe, f"--query-gpu={_GPU_QUERY}", "--format=csv,noheader,nounits"],
71                 capture_output=True,
72                 text=True,
73                 timeout=timeout,
74                 check=True,
75             ).stdout.strip()
76         except (subprocess.SubprocessError, OSError):
77             return None
78         row = out.splitlines()[0] if out else ""
79         parts = [p.strip() for p in row.split(",")]
80         if len(parts) < 7:
81             return None
82         vals = [_num(p) for p in parts[:7]]
83         temp, util, power, sm, sm_max, used, free = vals
84         throttled = bool(temp is not None and temp >= _THROTTLE_TEMP_C)

```

```

85     if sm is not None and sm_max:
86         throttled = throttled or sm < _THROTTLE_CLOCK_FRAC * sm_max
87     return {
88         "util_pct": util,
89         "temp_c": temp,
90         "power_w": power,
91         "sm_mhz": sm,
92         "sm_max_mhz": sm_max,
93         "vram_used_mb": used,
94         "vram_free_mb": free,
95         "throttled": throttled,
96     }
97
98
99     def system_stats() -> Optional[Dict[str, Any]]:
100         """CPU % + RAM via ``psutil``, or ``None`` if psutil isn't installed. Never
101         raises."""
102         try:
103             import psutil # optional dep (BSD-3); only used when present
104         except ImportError:
105             return None
106         try:
107             vm = psutil.virtual_memory()
108             return {
109                 "cpu_pct": psutil.cpu_percent(interval=None),
110                 "ram_used_gb": round(vm.used / 1e9, 2),
111                 "ram_total_gb": round(vm.total / 1e9, 2),
112                 "ram_pct": vm.percent,
113             }
114         except Exception: # noqa: BLE001 - psutil edge cases must not break the run
115             return None
116
117     def gpu_name() -> Optional[str]:
118         """The GPU model string (for the runlog header), or ``None`` without nvidia-smi."""
119         exe = shutil.which("nvidia-smi")
120         if not exe:
121             return None
122         try:
123             out = subprocess.run(
124                 [exe, "--query-gpu=name", "--format=csv,noheader"],
125                 capture_output=True,
126                 text=True,
127                 timeout=4.0,
128                 check=True,
129             ).stdout
130         except (subprocess.SubprocessError, OSError):
131             return None
132         return out.splitlines()[0].strip() if out else None
133
134
135     class RunTelemetry:
136         """A small rotating JSONL runlog of execution velocity + machine health.
137
138         Call ``snapshot(done, total, phase=...)`` from a progress callback and ``event(kind,
139         msg)`` at
140         milestones. The file holds a ``meta`` header line plus the most recent snapshot /
141         event lines
142         that fit within ``max_bytes`` (~10 KB), rewritten atomically on every write.
143         """
144
145         def __init__(
146             self,
147             path: str,
148             label: str = "run",

```

```

147         max_bytes: int = 10_000,
148         gpu_fn: GpuFn = gpu_stats,
149         sys_fn: SysFn = system_stats,
150         clock: Callable[[], float] = time.time,
151     ) -> None:
152         self.path = path
153         self.label = label
154         self.max_bytes = max_bytes
155         self._gpu_fn = gpu_fn
156         self._sys_fn = sys_fn
157         self._clock = clock
158         self._t0 = clock()
159         self._last_done: Optional[int] = None
160         self._last_t: Optional[float] = None
161         self._lines: List[
162             str
163         ] = [] # recent body lines (on-disk ring, trimmed to max_bytes)
164         self._meta = json.dumps(
165             {
166                 "meta": {
167                     "label": label,
168                     "started": self._iso(self._t0),
169                     "pid": os.getpid(),
170                     "gpu_name": gpu_name(),
171                     "cpu_count": os.cpu_count(),
172                 }
173             },
174             separators=(",", ":"),
175         )
176         self._write()
177
178     # -- public API -----
179     def snapshot(
180         self,
181         done: Optional[int] = None,
182         total: Optional[int] = None,
183         phase: Optional[str] = None,
184         **extra: Any,
185     ) -> Dict[str, Any]:
186         """Record a velocity + health sample; ``fps`` is derived from the previous
187         snapshot."""
188         now = self._clock()
189         rec: Dict[str, Any] = {
190             "t": self._iso(now),
191             "elapsed_s": round(now - self._t0, 1),
192         }
193         if phase is not None:
194             rec["phase"] = phase
195         if done is not None:
196             rec["done"] = done
197             if total:
198                 rec["total"] = total
199                 rec["pct"] = round(100.0 * done / total, 1)
200             if (
201                 self._last_done is not None
202                 and self._last_t is not None
203                 and now > self._last_t
204             ):
205                 rec["fps"] = round((done - self._last_done) / (now - self._last_t), 1)
206                 self._last_done, self._last_t = done, now
207             rec.update(extra)
208         self._attach_health(rec)
209         self._append(rec)
210         return rec

```

```

211     def event(self, kind: str, msg: str = "", **extra: Any) -> Dict[str, Any]:
212         """Record a milestone (start, video_done, error, restart) WITH a health
sample."""
213         now = self._clock()
214         rec: Dict[str, Any] = {
215             "t": self._iso(now),
216             "elapsed_s": round(now - self._t0, 1),
217             "event": kind,
218         }
219         if msg:
220             rec["msg"] = msg
221         rec.update(extra)
222         self._attach_health(rec)
223         self._append(rec)
224         return rec
225
226     # -- internals -----
227     def _attach_health(self, rec: Dict[str, Any]) -> None:
228         gpu = self._safe(self._gpu_fn)
229         sys_ = self._safe(self._sys_fn)
230         if gpu is not None:
231             rec["gpu"] = gpu
232         if sys_ is not None:
233             rec["sys"] = sys_
234
235     def _append(self, rec: Dict[str, Any]) -> None:
236         self._lines.append(json.dumps(rec, separators=(",", ":")))
237         budget = self.max_bytes - len(self._meta) - 1
238         while len(self._lines) > 1 and sum(len(s) + 1 for s in self._lines) > budget:
239             self._lines.pop(0) # drop oldest body line; the meta header is always kept
240         self._write()
241
242     def _write(self) -> None:
243         try:
244             tmp = self.path + ".tmp"
245             with open(tmp, "w", encoding="utf-8") as f:
246                 f.write(self._meta + "\n")
247                 for s in self._lines:
248                     f.write(s + "\n")
249             os.replace(tmp, self.path)
250         except OSError:
251             pass # telemetry must never break the run it observes
252
253     @staticmethod
254     def _safe(fn: GpuFn) -> Optional[Dict[str, Any]]:
255         try:
256             return fn()
257         except Exception: # noqa: BLE001 - a collector hiccup must not break the run
258             return None
259
260     @staticmethod
261     def _iso(t: float) -> str:
262         return time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(t))
263

```

11. assistant (2026-07-04T00:44:18.133Z)

Perfect! Now let me check the full diff details more carefully. Let me read the tool-results file:

12. user (2026-07-04T00:44:20.287Z)

```

diff --git a/SESSION_HANDOFF.md b/SESSION_HANDOFF.md
index 31ea4c14..a50cc8a1 100644
--- a/SESSION_HANDOFF.md

```

+++ b/SESSION_HANDOFF.md

@@ -2,49 +2,51 @@

You are here

-A6 is implemented and open for review as PR #455:

-`https://github.com/Khelsutra/badminton-highlight-indexer/pull/455`.

+A6 PR #455 is merged. This checkout is on `codex/a7-a9-quality-chain`, fast-forwarded to
+`origin/master` at `c5fc27c`, with tracked edits in:

-The branch is `codex/a6-ablation-cli`, rebased onto `origin/master` after A5

-merged as `c8eec7c` / PR #454. It fixes the `python -m backend.eval.ablation`

-circular import by making `ablation.export\_pdf` a lazy wrapper, adds a

-module-entrypoint regression test, and records the n=15 R2/tolF1 contrast in

-`docs/ALPHA\_LAUNCH\_READINESS.md`.

+ `backend/eval/fusion\_golden.py`

+ `backend/eval/fusion\_audio.py`

+ `tests/test\_fusion\_compare.py`

+ `tests/test\_audio\_features.py`

+ `docs/ALPHA\_LAUNCH\_READINESS.md`

+ `SESSION\_HANDOFF.md`

-Validation completed locally on 2026-07-03:

-`ruff check .`, `python -m mypy backend training`, and

-`python run\_all\_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`

-all passed. Full suite result: 1609 passed, 6 skipped, 85.65% coverage.

+A7/A8/A9 have been run locally against the 15-video corpus, and draft PR #458 is open:

+`https://github.com/Khelsutra/badminton-highlight-indexer/pull/458`. The generated

+feature/model artifacts live under ignored `output/` and `artifacts/` paths and should not
+be committed.

Next steps / open threads

-1. Review/merge PR #455 for A6.

-2. A7 is the next indexer-owned quality item: regenerate or upconvert

- full-fusion feature JSONs for the newer nine golden videos so fusion/person

- /audio analyses can run on all 15 videos. GPU is approved on this machine.

-3. A8 follows A7: rerun `fusion\_compare`, group ablation, and served contrast;

- explicitly promote or reject fusion/person/Gate-A served-default decisions.

-4. A3 is owned by another agent; do not duplicate that collector/vault work.

+1. Address review comments on draft PR #458; convert to ready only after owner/reviewer agrees
+ the A9 TemporalMaxer scope question is settled.

+2. Decide whether A9 is considered complete with the current Gen-0 non-serving shadow
+ bundle or whether TemporalMaxer needs a follow-up scorer-swap PR.

+3. Do not touch A1; another agent owns it.

+4. A3 is in collector PR #62 and records all 15 source videos; the two Boxhill originals
+ were verified on `F:\`.

Ramp-up kit

- `docs/ALPHA\_LAUNCH\_READINESS.md`

-- `docs/data\_pipeline/GOLDEN\_VIDEOS.md`

-- `docs/R2\_EVAL\_METRIC\_DESIGN.md`

-- `backend/eval/ablation.py`

-- `backend/eval/ablation\_pdf.py`

-- `backend/eval/rally\_seg\_eval.py`

-- `tests/test\_ablation.py`

-- `output/human\_lovo\_manifest.json`

+ `backend/eval/fusion\_golden.py`

+ `backend/eval/fusion\_audio.py`

+ `training/gen0/harness.py`

+ `eval\_baselines/heuristic\_lovo\_n15.json`

+ `output/a7\_remaining\_default.out.log`

+ `output/a8\_audio\_augment.out.log`

+ `output/a6\_r2\_ablation.json`

```

+- `artifacts/rally_tal_gen0/0.1.1-n15-shadow.20260704/manifest.json`

## Key decisions

-- R2/tolF1 is supported for the baseline-off to served milestone contrast on
- the 15-video corpus: `Delta tolF1 = +0.225`, CI `[+0.165, +0.283]`,
- sign `14/0`, `p = 0.000`.
-- This does not approve fusion/person/audio default changes. Current full-fusion
- ablation still runs on only six videos because the newer nine do not yet have
- full-fusion feature schema.
-- Branching discipline remains important: always fetch/pull first, branch from
- `origin/master`, stage explicit paths, and verify `origin/master..HEAD` before
- pushing.
+- A7: full-fusion schema and audio features are present locally for all 15 videos.
+- A8: person fusion and audio are suggestive but not statistically clean; do not promote
+ either as a default. Gate A regresses under served windowing; keep it opt-in/analysis-only.
+- R2/cap/inactivity remains the strongest quality result on n15: F1 `0.237 -> 0.472`,
+ tolF1 `0.138 -> 0.364`, merge rate `0.638 -> 0.135`; net over-seg guard passes.
+- A9: Gen-0 LogReg/TAL n15 shadow bundle beats the heuristic mean (`0.551 > 0.4456`)
+ but is non-serving (`9/15` paired wins, `p=0.3036`; #172 and WASB C3 still block).
+- Branching discipline remains important: fetch/pull first, stage explicit paths, and
+ verify `origin/master..HEAD` contains only this branch's commits before pushing.
diff --git a/backend/eval/fusion_audio.py b/backend/eval/fusion_audio.py
index b5c65ec4..7dff0c94 100644
--- a/backend/eval/fusion_audio.py
+++ b/backend/eval/fusion_audio.py
@@ -17,14 +17,17 @@
 import json
 import logging
 import os
-from typing import Any, Dict, List
+from typing import Any, Dict, List, Optional, Sequence

 from backend.eval import experiment
 from backend.eval.fusion_compare import delta_stats
 from backend.eval.fusion_golden import (
     PERSON_FEATURE_NAMES,
     SHUTTLE_FEATURE_NAMES,
+     SOURCE_MANIFEST_ENV,
     _derive_video_path,
+     _filter_specs,
+     attach_source_video_paths,
 )
 from backend.eval.golden_manifest import load_resolved_manifest
 from backend.pipeline.audio.hit_detector import detect_hits_in_video
@@ -33,9 +36,24 @@
 logger = logging.getLogger(__name__)

-def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
+def augment(
+    manifest_path: str,
+    features_dir: str,
+    k: float = 2.5,
+    source_manifest: Optional[str] = None,
+    only: Sequence[str] = (),
+) -> List[str]:
     """Add the 3 audio features to each existing `<name>_golden_features.json` window."""
     specs = load_resolved_manifest(manifest_path)
+    source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
+    if source_manifest:
+        updated = attach_source_video_paths(specs, source_manifest)
+        print(
+            f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
+            f"from {source_manifest}",

```

```

+         flush=True,
+     )
+     specs = _filter_specs(specs, only)
+     written: List[str] = []
+     for spec in specs:
+         jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
@@ -45,6 +63,10 @@ def augment(manifest_path: str, features_dir: str, k: float = 2.5) ->
List[str]:
+         )
+         continue
+         video = _derive_video_path(spec)
+         if not os.path.isfile(video):
+             raise FileNotFoundError(
+                 f"{spec.get('name')}: source video not found for audio augmentation: {video}"
+             )
+         print(
+             f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
+             flush=True,
@@ -129,6 +151,21 @@ def main() -> None:
+     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
+     ap.add_argument("--features-dir", default="output")
+     ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
+     ap.add_argument(
+         "--source-manifest",
+         default=None,
+         help=(
+             "collector golden_source_videos.json; overrides video paths for source/proxy "
+             f"videos (or set {SOURCE_MANIFEST_ENV})"
+         ),
+     )
+     ap.add_argument(
+         "--only",
+         action="append",
+         default=[],
+         metavar="VIDEO_ID",
+         help="augment one manifest video by name; repeatable",
+     )
+     ap.add_argument(
+         "--augment",
+         action="store_true",
@@ -142,7 +179,13 @@ def main() -> None:
+     args = ap.parse_args()

+     if args.augment:
+         augment(args.manifest, args.features_dir, args.k)
+         augment(
+             args.manifest,
+             args.features_dir,
+             args.k,
+             source_manifest=args.source_manifest,
+             only=args.only,
+         )
+     if args.compare:
+         videos = load_videos_with_audio(args.features_dir)
+         if len(videos) < 2:
diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
index aee6d7e5..0666a0ca 100644
--- a/backend/eval/fusion_golden.py
+++ b/backend/eval/fusion_golden.py
@@ -21,7 +21,7 @@
import json
import os
import time
from typing import Any, Dict, List, Optional
from typing import Any, Dict, List, Optional, Sequence

```

```
from backend.eval import distill_local
from backend.eval.calibrate_wasb import load_golden_gts
```

13. assistant (2026-07-04T00:44:22.016Z)

Let me continue reading the diff:

14. user (2026-07-04T00:44:24.580Z)

<persisted-output>

Output too large (37.3KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\tool-results\bgxrcr1ro.txt

Preview (first 2KB):

```
from backend.eval.calibrate_wasb import load_golden_gts
@@ -36,6 +36,100 @@
SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
# The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
+SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
+
+
+def is_full_fusion_feature_file(path: str) -> bool:
+    """True when an existing golden feature JSON already has shuttle+person cue dicts.
+
+    Corpus-growth runs can leave leaner windowing-only JSONs with the same filename pattern.
+    Those are useful for gts/windowing analysis, but they are not the replayable fusion
+    substrate that ``fusion_compare`` and ``ablation`` need.
+
+    """
+
+    try:
+        with open(path, encoding="utf-8") as f:
+            data = json.load(f)
+    except (OSError, json.JSONDecodeError, TypeError):
+        return False
+    candidates = data.get("candidates")
+    if not isinstance(candidates, list):
+        return False
+    if not candidates:
+        return True
+    return all(
+        isinstance(candidate, dict)
+        and "shuttle" in candidate
+        and "person" in candidate
+        for candidate in candidates
+    )
+
+def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
+    """Load ``video_id -> local_path`` from the collector golden-source manifest.
+
+    The collector records source/proxy ownership and full-file MD5s. Only entries marked
+    ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
+
+    """
+
+    if not source_manifest_path:
+        return {}
+    with open(source_manifest_path, encoding="utf-8") as f:
+        data = json.load(f)
+    entries = data.get("videos", data) if isinstance(data, dict) else data
+    if not isinstance(entries, list):
+        raise ValueError(f"source manifest must contain a videos list: {source_manifest_path}")
+
+    paths: Dict[str, str] = {}
```

```
+     for entry in entries:
+ ...
+ </persisted-output>
```

15. assistant (2026-07-04T00:44:26.507Z)

Let me read the rest of the fusion_golden diff more systematically:

16. user (2026-07-04T00:44:28.219Z)

```
42: def is_full_fusion_feature_file(path: str) -> bool:
68: def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
97: def attach_source_video_paths(
120: def _filter_specs(
135: def _derive_video_path(spec: Dict[str, Any]) -> str:
148: def extract_video(
201:     def _progress(done: int, total: int) -> None:
269: def run(
385: def main() -> None:
```

17. assistant (2026-07-04T00:44:46.348Z)

Now let me analyze each critical area identified in the angle:

****ANALYSIS:****

1. `is\_full\_fusion\_feature\_file` (lines 42-65)

Looking at line 58-59:

```
```python
if not candidates:
 return True
```
```

This is a ****critical bug****. When candidates is an empty list `[]`:

- The file passed the JSON load (not corrupted)
- The candidates list exists (not None)
- But the list is empty

An empty candidates list indicates an ****incomplete or stale extraction**** (windowing completed without finding candidates), NOT a successful full-fusion extraction. The function returns `True`, classifying it as "complete," which causes `run()`'s resume logic to SKIP re-extracting it.

****Failure scenario:**** A crashed extraction that wrote only `{"candidates": []}` will be classified as fully extracted ? permanently skipped on resume ? stale file kept forever.

Also at lines 60-65, the check only verifies key *presence* (`"shuttle" in candidate`), not that values are non-null:

```
```python
and "shuttle" in candidate
and "person" in candidate
```
```

A candidate with `{"shuttle": null, "person": null}` would pass this check ? classified as full-fusion.

2. `run()` resume logic (lines 322-337)

At lines 322-329 with `missing\_fusion\_only=True, fresh=False, existing=True, existing\_full=True`:

- Condition is True ? continues (skips extraction)
- Correct behavior: skip already-done work

At lines 330-337 with `missing_fusion_only=False, fresh=False, existing=True,

```
existing_full=False`:
- Line 338-343 re-extracts a windowing-only JSON (correct)
```

But consider: `missing\_fusion\_only=False, fresh=False, existing=True, existing\_full=True` (lines 330-337):

- Enters the branch ? skips extraction
- Correct behavior: skip complete work

The logic appears sound IF `is\_full\_fusion\_feature\_file()` correctly identifies full-fusion. But because of issue #1, this is unreliable.

3. `\_progress` & telemetry snapshot (lines 201-212)

```
At line 204:
```python
rec = telemetry.snapshot(done, total, phase=spec["name"])
```
```

Looking at `run\_telemetry.py` lines 179-209, `snapshot()` **always** returns a dict ? specifically `rec: Dict[str, Any]` populated with `t`, `elapsed\_s`, and health data via `\_attach\_health()` (lines 227-233).

```
At line 206:
```python
if temp is not None and float(temp) >= max_gpu_temp_c:
```
```

The `\_attach\_health()` method (line 227-233) calls `gpu\_stats()` and attaches it to the record with key `"gpu"`. The `rec.get("gpu")` should work fine.

However, if `telemetry is None` (line 203 check), then `rec is None` ? the condition `if max\_gpu\_temp\_c is not None and rec is not None:` at line 205 correctly guards the safety check. The feature is NOT dead, but it will silently skip thermal checks if telemetry fails to initialize.

Actually, reviewing more carefully: the thermal abort check is **safe**. If telemetry is None, the guard at line 205 prevents accessing `rec`. If telemetry is present, it always returns a dict with "gpu" attached if available.

4. `load\_source\_video\_paths` & `attach\_source\_video\_paths` (lines 68-117)

```
At line 91:
```python
if entry.get("source_status", "present_local") != "present_local":
 continue
```
```

An entry with NO `source\_status` key defaults to `"present\_local"` and is INCLUDED. This means:

- Entries without a status field are treated as "local present"
- The path is attached unconditionally

This is a **design choice** that could be correct (assume entries without status are local) or incorrect (missing status should be ignored). Without seeing the actual source manifest schema, this is **ambiguous but explicit**.

The mutation of `spec` dicts in place (line 115 `spec["video\_path"] = source\_path`) modifies the input, which is expected given the function name `attach\_source\_video\_paths`. This is correct.

5. `fusion\_audio.augment` FileNotFoundError (lines 66-69)

```
```python
if not os.path.isfile(video):
 raise FileNotFoundError(...)
```
```

This is ****INSIDE the per-spec loop**** (line 58). One missing video aborts the entire batch after partial work. The function returns after partial writes, leaving the output directory in an inconsistent state.

****Failure scenario:**** 8 videos augmented successfully, then spec #9's video is missing ? raises, exits, leaving 8 augmented JSONs + 7 unaugmented. Rerunning must use `--only` to skip the 8 already done, or they get re-processed.

This is a ****footgun****: ideally, it should either skip-and-continue (log a warning) or collect errors and fail at the end after processing all specs.

6. `_filter_specs` dedup + missing check (lines 120-132)

```
```python
wanted = list(dict.fromkeys(only))
...
filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
...
missing = [name for name in wanted if name not in found]
if missing:
 raise ValueError(f"--only requested unknown video(s): {' '.join(missing)}")
...

- Line 125: `dict.fromkeys(only)` deduplicates while preserving order (correct)
- Line 129: missing check is computed correctly
- Raises if any requested video is not found (expected behavior)
```

**\*\*This is correct.\*\***

---

Let me verify the specific lines and compile findings:

## 18. user (2026-07-04T00:44:47.753Z)

```
if not candidates:
 return True
return all(
 isinstance(candidate, dict)
 and "shuttle" in candidate
 and "person" in candidate
 for candidate in candidates
)

if entry.get("source_status", "present_local") != "present_local":
 rec = telemetry.snapshot(done, total, phase=spec["name"])
if max_gpu_temp_c is not None and rec is not None:
 gpu = rec.get("gpu") or {}
 temp = gpu.get("temp_c")
 if temp is not None and float(temp) >= max_gpu_temp_c:
 raise RuntimeError(
 f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
 f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal shutdown"
)
if missing_fusion_only and not fresh and existing_full:
 print(
 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
 f"features already present -> {out}",
 flush=True,
)
 written.append(out)
 continue
if not fresh and existing and existing_full:
 print(
 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted "
 f"-> {out} (use --fresh to redo)",
```

```

 flush=True,
)
 written.append(out)
 continue

```

### 19. assistant (2026-07-04T00:44:49.812Z)

Perfect. Now let me check the fusion\_audio lines more carefully:

### 20. user (2026-07-04T00:44:51.090Z)

```

for spec in specs:
 jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
 if not os.path.isfile(jp):
 print(
 f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden first;
skipping"
)
 continue
 video = _derive_video_path(spec)
 if not os.path.isfile(video):
 raise FileNotFoundError(
 f"{spec.get('name')}: source video not found for audio augmentation: {video}"
)
 print(

```

### 21. assistant (2026-07-04T00:45:02.150Z)

Excellent. Now let me verify the actual line numbers by examining the context in the extracted file I already have:

Looking at `pr458\_fusion\_audio.py`:

- Lines 58-69 show the loop with the `FileNotFoundError` inside the per-spec loop

Looking at `pr458\_fusion\_golden.py`:

- Lines 58-59: `if not candidates: return True` (the empty-list bug)
- Lines 60-65: the all() check
- Line 91: the `entry.get("source\_status", "present\_local")` default logic
- Lines 322-337: the resume logic with missing\_fusion\_only
- Lines 204-212: the telemetry snapshot and thermal check

Now I'll compile the findings:

```

```json
[
  {
    "file": "backend/eval/fusion_golden.py",
    "line": 59,
    "summary": "is_full_fusion_feature_file returns True for empty candidates list,
misclassifying incomplete extractions as complete",
    "failure_scenario": "A crashed extraction that wrote {\"candidates\": []} will be classified
as fully extracted, permanently skipped on resume, leaving stale empty file"
  },
  {
    "file": "backend/eval/fusion_golden.py",
    "line": 62,
    "summary": "Key presence check ('shuttle' in candidate) does not verify value is non-null,
allowing null-valued cue dicts to pass full-fusion classification",
    "failure_scenario": "A candidate with {\"shuttle\": null, \"person\": null} passes
is_full_fusion_feature_file() ? resume skips re-extraction ? fusion_compare gets null feature
values"
  },
  {
    "file": "backend/eval/fusion_audio.py",

```

```

    "line": 67,
    "summary": "FileNotFoundError raised inside per-spec loop aborts batch after partial writes,
leaving output directory inconsistent",
    "failure_scenario": "8 videos successfully augmented, spec #9 video missing ? raises, exits
? 8 augmented JSONs + 7 unaugmented remain; rerun must use --only or will re-process"
},
{
    "file": "backend/eval/fusion_golden.py",
    "line": 91,
    "summary": "entry.get('source_status', 'present_local') defaults MISSING status to
'present_local', auto-attaching video path for entries with no recorded status",
    "failure_scenario": "Source manifest entry without source_status key is treated as
'present_local' and path attached; if status was intentionally omitted (e.g., to mark as
untested), path is incorrectly used"
}
]
...

```

Wait, let me reconsider #3 (telemetry). Looking again at lines 203-206:

```

```python
rec = None
if telemetry is not None:
 rec = telemetry.snapshot(done, total, phase=spec["name"])
if max_gpu_temp_c is not None and rec is not None:
...

```

Actually, `rec = None` is the default, and it's only set if `telemetry is not None`. The guard `rec is not None` protects the access, so the thermal abort is **safe and working correctly**. This is NOT a bug.

Let me reconsider the actual issues more carefully. Actually, re-reading the code at lines 201-212 in the extracted file:

```

```python
def _progress(done: int, total: int) -> None:
    rec = None
    if telemetry is not None:
        rec = telemetry.snapshot(done, total, phase=spec["name"])
    if max_gpu_temp_c is not None and rec is not None:
        gpu = rec.get("gpu") or {}
        temp = gpu.get("temp_c")
...

```

Wait - I see `rec = None` at line 202, but in the diff I don't see that. Let me check the actual PR branch file again:

22. user (2026-07-04T00:45:03.379Z)

```

def _progress(done: int, total: int) -> None:
    rec = None
    if telemetry is not None:
        rec = telemetry.snapshot(done, total, phase=spec["name"])
    if max_gpu_temp_c is not None and rec is not None:
        gpu = rec.get("gpu") or {}
        temp = gpu.get("temp_c")
        if temp is not None and float(temp) >= max_gpu_temp_c:
            raise RuntimeError(
                f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
                f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal shutdown"
            )

```

23. assistant (2026-07-04T00:45:22.836Z)

Good, `rec = None` is initialized at line 202. The telemetry logic is **safe**. Let me remove

that from my findings.

Actually, looking at issue #4 more carefully: the default of `entry.get("source\_status", "present\_local")` when the key is missing? is this actually problematic? Let me re-read the docstring:

```
```python
def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
 """Load ``video_id -> local_path`` from the collector golden-source manifest.

 The collector records source/proxy ownership and full-file MD5s. Only entries marked
 ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
 """
...```
```

The intent is "only entries marked present\_local". But the code defaults missing status to present\_local. This is a design bug IF missing status should mean "unknown" (not "present\_local"). However, it could also be intentional: "assume local if no status recorded."

Given the ambiguity and that it's a design decision (not a logic error per se), I'll keep it but note it's questionable.

Let me finalize my candidate list focusing on concrete correctness bugs:

```
```json
[
  {
    "file": "backend/eval/fusion_golden.py",
    "line": 59,
    "summary": "is_full_fusion_feature_file returns True for empty candidates list,
misclassifying incomplete/crashed extractions as complete",
    "failure_scenario": "Extraction crashes mid-processing and writes {\"candidates\": []};
function returns True; run() resumes and skips re-extraction; stale empty file kept forever"
  },
  {
    "file": "backend/eval/fusion_golden.py",
    "line": 62,
    "summary": "Key presence check ('shuttle' in candidate) does not verify values are non-null,
allowing null-valued cue dicts to pass full-fusion classification",
    "failure_scenario": "Candidate with {\"shuttle\": null, \"person\": null} passes
is_full_fusion_feature_file() check; resume skips re-extraction; fusion_compare receives null
feature values causing NaN/errors"
  },
  {
    "file": "backend/eval/fusion_audio.py",
    "line": 67,
    "summary": "FileNotFoundError raised inside per-spec loop aborts batch after partial writes,
leaving output in inconsistent state",
    "failure_scenario": "8 of 10 videos augmented successfully; spec #9 video file missing;
raises and exits; 8 modified JSONs + 7 unmodified JSONs remain; rerun requires --only or re-
processes completed videos"
  },
  {
    "file": "backend/eval/fusion_golden.py",
    "line": 91,
    "summary": "entry.get('source_status', 'present_local') defaults missing status to
'present_local', auto-including entries with no recorded source status",
    "failure_scenario": "Source manifest entry lacks source_status key (intended to mean
unknown/untested); code treats it as present_local and attaches path; incorrect path used for
extraction"
  }
]
...```
```

Actually, on reconsideration of #4: if the source manifest is authoritative and a missing key

means "not specified in this manifest run," then defaulting to present_local is **reasonable** (assume if not marked otherwise, it's local). This is more of a design question than a bug. Let me check if there are any other concrete issues I might have missed.

Let me also reconsider the `all()` check at lines 60-65. The check is:

```
```python
return all(
 isinstance(candidate, dict)
 and "shuttle" in candidate
 and "person" in candidate
 for candidate in candidates
)
```
```

This checks:

1. candidate is a dict ?
2. "shuttle" key exists ?
3. "person" key exists ?

But it does NOT check that the values are truthy or non-empty. So `{"shuttle": null, "person": null}` or `{"shuttle": {}, "person": {}}` would pass.

Is this a bug? Looking at the `extract\_video()` function (lines 251-259), it creates candidates with:

```
```python
"shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
"person": {k: float(v) for k, v in person.items()},
```
```

So shuttle and person should be non-empty dicts. If they're null, that's a data corruption/incomplete write, which the check doesn't catch.

Actually, maybe the right fix is to check that both keys exist AND their values are dicts (not null). Let me see if there's a way a valid full-fusion file could have null values. Looking at extract_video, it always creates non-null dicts. So null values = corruption or incomplete write.

The check should be:

```
```python
return all(
 isinstance(candidate, dict)
 and isinstance(candidate.get("shuttle"), dict)
 and isinstance(candidate.get("person"), dict)
 for candidate in candidates
)
```
```

OK, so issue #2 is valid: the key-presence check is insufficient.

Now let me reconsider if issue #4 is a bug or not. Looking at the code flow:

```
```python
for entry in entries:
 if not isinstance(entry, dict):
 continue
 video_id = entry.get("video_id") or entry.get("name")
 local_path = entry.get("local_path")
 if not video_id or not local_path:
 continue
 if entry.get("source_status", "present_local") != "present_local":
 continue
 paths[str(video_id)] = str(local_path)
```
```

The default of `"present\_local"` when source_status is missing means: **include entries that**

have no explicit status**. This could be intentional (assume local unless proven otherwise) or a bug (should skip entries without explicit status).

Without more context about the source manifest schema, I can't definitively call this a bug. But it IS suspicious: why default to an active status rather than a neutral skip? I'll include it but note it's design-questionable.

Actually, re-reading the docstring: "Only entries marked ``present\_local``". The word "marked" suggests explicit marking. So defaulting an unmarked entry to that status is **contradictory to the docstring**. I'll keep this as a finding.

Let me finalize:

```
```json
[
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 59,
 "summary": "is_full_fusion_feature_file returns True for empty candidates list, misclassifying incomplete/crashed extractions as complete",
 "failure_scenario": "Extraction crashes mid-processing, writes {\"candidates\": []}; function returns True; run() resume skips re-extraction; stale empty file permanently kept"
 },
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 62,
 "summary": "Key-presence check does not validate that shuttle/person values are non-null dicts, allowing corrupted entries to pass full-fusion classification",
 "failure_scenario": "Candidate with {\"shuttle\": null, \"person\": null} (partial/crashed write) passes is_full_fusion_feature_file(); resume skips re-extraction; fusion_compare reads null values causing NaN/errors"
 },
 {
 "file": "backend/eval/fusion_audio.py",
 "line": 67,
 "summary": "FileNotFoundError raised inside per-spec loop aborts batch after partial writes, leaving output directory inconsistent",
 "failure_scenario": "8 of 10 videos augmented successfully; spec #9 video missing; raises and exits; 8 modified JSONs remain alongside 7 unmodified; rerun either re-processes completed videos or requires manual --only filtering"
 },
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 91,
 "summary": "entry.get('source_status', 'present_local') defaults unmarked entries to 'present_local', contradicting docstring requirement of explicit marking",
 "failure_scenario": "Source manifest entry lacks source_status key; code treats as present_local and attaches path; if unmarked meant untested/unavailable, incorrect path used for extraction"
 }
]
```
```

This is my final analysis. All four issues are concrete and real.

```
```json
[
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 59,
 "summary": "is_full_fusion_feature_file returns True for empty candidates list, misclassifying incomplete/crashed extractions as complete",
 "failure_scenario": "Extraction crashes mid-processing, writes {\"candidates\": []}; function returns True; run() resume skips re-extraction; stale empty file permanently kept"
 },

```

```

{
 "file": "backend/eval/fusion_golden.py",
 "line": 62,
 "summary": "Key-presence check does not validate that shuttle/person values are non-null
dicts, allowing corrupted entries to pass full-fusion classification",
 "failure_scenario": "Candidate with {\"shuttle\": null, \"person\": null} (partial/crashed
write) passes is_full_fusion_feature_file(); resume skips re-extraction; fusion_compare reads
null values causing NaN/errors"
},
{
 "file": "backend/eval/fusion_audio.py",
 "line": 67,
 "summary": "FileNotFoundError raised inside per-spec loop aborts batch after partial writes,
leaving output directory inconsistent",
 "failure_scenario": "8 of 10 videos augmented successfully; spec #9 video missing; raises
and exits; 8 modified JSONs remain alongside 7 unmodified; rerun either re-processes completed
videos or requires manual --only filtering"
},
{
 "file": "backend/eval/fusion_golden.py",
 "line": 91,
 "summary": "entry.get('source_status', 'present_local') defaults unmarked entries to
'present_local', contradicting docstring requirement of explicit marking",
 "failure_scenario": "Source manifest entry lacks source_status key; code treats as
present_local and attaches path; if unmarked meant untested/unavailable, incorrect path used for
extraction"
}
]
...

```